Week 27: Graded Mini Project

Project title: Policy Assistant Agent (Single + Memory + Graph)

Objective:
Apply your understanding of agentic AI systems, tool-calling, graph orchestration, and lightweight memory to design and evaluate a grounded Policy Assistant. Build an agent that answers employee questions using a small, local policy corpus, plans and adapts through tool use, cites document IDs, persists only verified facts into memory, and operates deterministically under explicit budget and stop constraints. Evaluate grounding, citation accuracy, recovery behaviour, and memory effectiveness using acceptance tests and a simple critic.

Submission Guidelines

Ensure your submission includes:

- A modular code repository implementing the Policy Assistant Agent
- Local policy dataset (CSV or JSONL) and FAISS retrieval setup
- Tool implementations (policy_search, read_policy, optional calculator)
- Agent logic, graph orchestration, and memory integration
- Evaluation scripts, result tables, logs, and screenshots or short demo video
- README with setup instructions, design choices, and limitations

To submit your response:

- Click on the Start Assignment button
- Upload the complete project files
- Click Submit Assignment after uploading
- Save your work as PDF named Week27_Graded Mini Project_[Your Last name].pdf

Your submission will be considered complete ONLY if it follows the required template, addresses all project tasks clearly, meets acceptance criteria, and is submitted on time in the correct format.

Note: This is a required activity and counts toward programme completion.

Reflect on the tasks and respond to the following questions.

Part 1: Foundations

Task 1.1: Data prep

- Create policies.jsonl with fields: doc_id, text, date.

What I built: data/policies.jsonl, a local corpus of 10 internal policies (remote work, annual leave, travel and expenses, IT security, parental leave, code of conduct, data retention, overtime, hardware refresh, learning budget). Each line is one JSON object with doc_id, text and date. **Design choices:** doc_ids follow a name_vN pattern (e.g. remote_work_v3) so they can double as citation tokens in answers. Each policy holds 3–4 short, rule-style sentences with exact numbers (3 days per week, INR 1,500, 24 days), so answers can be checked by quoting. **Validation (models.py):** a pydantic Policy schema enforces the doc_id format, ISO dates (YYYY-MM-DD) and non-empty text, and load_policies() rejects duplicate ids. **Result:** all 10 rows load and validate. Three deliberately bad rows (bad id format, bad date, text too

For learner use only

short) are rejected, and the facts the starter queries need are confirmed present.

Task 1.2: Tools

- Implement policy_search(query, k) using a FAISS retriever.
- Implement read_policy(doc_id) to return full text; raise a typed error when missing.
- *(Optional)* Add a calculator tool with safe division.

policy_search(query, k): each policy is embedded with OpenAI text-embedding-3-small and stored in a LangChain FAISS index (inner product on normalised vectors, i.e. cosine similarity). It returns the top-k hits as {doc_id, date, score, snippet}. The correct policy ranked first for all starter questions, with a clear margin: remote work 0.61 vs 0.34, meals cap 0.64 vs 0.38, MFA/VPN 0.60 vs 0.25.

read_policy(doc_id): returns the full policy, or raises a typed PolicyNotFoundError (code POLICY_NOT_FOUND) carrying the bad id and a hint: "Call policy_search with the topic to find the correct doc_id."

Tool contracts and soft failure: the plain functions raise typed errors. The LangChain @tool wrappers catch them and return JSON {ok: false, error: {...}}, so a bad id never crashes the agent; it can read the error and recover.

calculator (optional): parses the expression with Python's AST, allowing only numbers and + − × ÷ // % **. Function calls, names and exponents above 10 are blocked, and division by zero raises CalculatorError. Verified: 12 * 4 = 48; 10/0, __import__('os') and 2**1000 are all rejected.

Task 1.3: Prompts

Create a system prompt with acceptance rules:

- doc_id token requirement
- Quoting requirement
- Step limits

System prompt (prompts.py): it defines six acceptance rules, and an answer that breaks any of them is rejected.

R1 Grounding: use only facts from tool outputs, never outside knowledge. If the answer is missing, reply exactly "I could not find this in the policy corpus."

R2 doc_id token: every final answer must cite each policy used as a bracketed token, e.g. [remote_work_v3], and only ids returned by a tool.

R3 Quoting: every exact rule (number, limit, requirement) needs a verbatim quote of at most 30 words, copied from read_policy output, in double quotes.

R4 Step limits: at most max_iterations (6) reasoning steps, at most 2 searches and 2 reads, and always read before quoting (snippets are not enough).

R5 Recovery and R6 Memory: on POLICY_NOT_FOUND, do not retry the same id; search instead. A memory hint is a lead to re-verify, not evidence. After testing, I added to R6 that the hint covers only the policy fact, so every part of the question must still be answered.

Fixed format (Answer: ... [doc_id] / Quote: "..." [doc_id]) makes the rules machine-checkable with regexes, and the critic and eval reuse those checks. All five required rules were verified present (PASS).

For learner use only

Task 1.4: Agent

- Build a deterministic ReAct-style tool-calling agent with verbose tracing.
- Implement “bad id” recovery: fail softly, then search.

Agent (agent.py): PolicyAgent is a ReAct executor around gpt-4o-mini (temperature 0, seed 42, tools bound). Each step sends the system prompt plus the conversation, runs the tool calls the model makes, appends the observations, and repeats until a final answer or max_iterations (6).

Verbose tracing: every step prints and logs Thought/Action/Observation with per-tool latency. Example trace: policy_search('remote work limit') → read_policy('remote_work_v3') → answer citing [remote_work_v3] with a verbatim quote (3 steps, 2 tool calls).

Bad-id recovery: for "Read policy remote_work_v9.9..." the trace was read_policy ERROR (POLICY_NOT_FOUND) → policy_search('remote work') → read_policy('remote_work_v3') ok → a correct cited answer. The error fails softly and the agent searches.

Other checks: multi-tool (calculator 48 plus the meals cap, with two tool calls in one step); a clean stop with stopped_reason=max_iterations when max_iterations=1; and identical answers on repeated runs (temperature 0 + seed). A deterministic offline model with the same tool-calling protocol is used for the unit tests.

Observation: the model recovered correctly but did not tell the user the requested id was missing, as rule R5 asks. The prompt alone does not guarantee this.

Task 1.5: Evaluation Version 1

- Create 6–10 canned questions.
- Print latency, steps, has_docid, and pass/fail.
- Save logs.

Evaluation (eval.py): 8 canned questions, namely the 6 starter queries (including bad-id recovery and calculator + policy) plus overtime and learning-budget questions. For each it records latency, steps, tool calls, has_docid, has_quote and pass/fail.

Pass rule: the expected doc_id is cited, every quote appears verbatim in the cited policy, the key facts are present (e.g. "3 days", "1,500"), and the agent stays within max_iterations.

Result (v1, agent only): 8/8 passed (100%; target ≥ 80%). Average latency 5.6 s; 3 steps and 2 tool calls per question, except recovery (4 steps, 3 calls) and multi-tool (3 calls).

Logs: each run saves logs/eval_v1_<time>.log (step and latency lines), a .jsonl with the full answer and tool trace per question, and results/eval_v1_<time>.csv (the table).

Part 2 — Memory + Graph + Critic + Polishing

Task 2.1 Memory

- Implement a FAISS store for verified facts; write only when doc_id is present.
- Read the “cheapest sufficient” entry first; pass hints into the question.

Memory (memory.py): a FAISS index of verified facts, keyed by the embedded question. facts.jsonl is the human-readable source of truth, and the index is rebuilt from it, so no pickle files are loaded.

Write only verified facts: a fact is stored only if the critic marked the answer valid, it cites a [doc_id] that exists, and its quote appears verbatim in that policy. Tested: no doc_id, fake quote, unknown id and critic-invalid writes were all rejected with the reason; only the two verified answers were stored.

For learner use only

Cheapest-sufficient read: first an exact question match (no embedding call), then a FAISS match with cosine ≥ 0.80, choosing the shortest of the qualifying entries; otherwise a miss and the tools are used. A paraphrase scored 0.80 and returned the 123-character entry instead of the 284-character one.

Hints: a hit becomes a hint appended to the question. The agent re-verifies with read_policy and skips policy_search: 1 tool call instead of 2, 2.9 s instead of 5.7 s.

Limit found: "How many days can I work from home?" fell below 0.80 and missed, so the threshold trades recall for safety.

Task 2.2: Graph orchestration

- Define nodes: plan → memory_read → agent_answer → critic → memory_write → stop.
- Implement a router with a budget and a single retry if the critic fails.

Graph (graph.py): a LangGraph StateGraph with explicit nodes plan → memory_read → agent_answer → critic → memory_write → stop, plus small routing nodes (retry, critic_failed, budget_stop). The state carries the question, plan, hint, answer, tool trace, critic result, steps, retries and stop_reason.

Budget: every counted node adds 1 to steps. The routers send the run to stop as soon as steps ≥ max_steps (8), with stop_reason budget_exhausted. Tested with max_steps=3: the run stopped after agent_answer (3/3), and the critic and memory write never ran.

Single retry: after the critic, a valid answer goes to memory_write. An invalid one re-runs agent_answer once, with the critic's issues as feedback; a second failure stops with critic_failed and nothing is written. Both paths were verified (retry then pass; retry then critic_failed, no write).

Results (eval v2): cold run 8/8, 8.1 s average, 2.25 tool calls, 0/8 memory hits. Warm run 7/8 at first, 4.9 s, 1.25 calls, 8/8 hits: Q6 dropped the calculation because the hint steered the model. After the R6 prompt fix, the warm run passed 8/8 at 5.2 s and 1.5 calls. Q2 warm needed one retry, the retry path working on a real case.

Observability: each node is wrapped to log its step and latency, and to turn exceptions into a soft stop.

Task 2.3: Critic

- Implement a minimal JSON output: {valid: bool, issues: []}.
- Check for doc_id presence and simple claims.

Critic (critic.py): returns minimal JSON {valid: bool, issues: []}. Four deterministic checks always run. C1: a [doc_id] token is present. C2: the cited id exists and was actually read with read_policy in this run. C3: every quote appears verbatim in the cited policy. C4, the simple-claims check: every number in the answer appears in the cited policy, the question or a calculator result.

LLM critic: with a key, gpt-4o-mini in JSON mode also judges the answer against the evidence. The answer is valid only if both pass, and an API error falls back to the rules, so the run never crashes.

Tests: a correct answer gives {valid: true, issues: []}. Each broken answer is caught by its check (missing id, unknown id, fake quote, wrong number). For "up to 4 days" the rules flagged C4 ['4'] and the LLM added: "the answer states 'up to 4 days per week' which contradicts the evidence that states 'up to 3 days per week'". 48 in the multi-tool answer was accepted because it came from the calculator.

Gap found: the critic checks what is in the answer, not what is missing. The Q6 warm answer that

For learner use only

omitted the calculation passed the critic; the eval's keyword check caught it.

Task 2.4: Tests + docs

- Add PyTests for tools, acceptance, and budget stop.
- Prepare README, run scripts, and screenshots.
- *(Optional)* Use SQLite checkpointing instead of in-memory storage.

Tests (pytest, 34 passed in 12.7 s): test_tools.py covers search top hits, the typed PolicyNotFoundError, the soft tool error and calculator safety. test_agent_acceptance.py covers that every canned query is cited, quoted and critic-valid, the bad-id recovery order, multi-tool, the max_iterations stop, the ≥ 80% pass rate, cold vs warm memory, and verified-only memory writes. test_graph_budget.py covers the full node path, the budget stop, exactly one retry, critic_failed with no write, the warm memory hit and SQLite checkpointing.

Determinism: the tests use the offline stand-in model, so they need no API key, cost nothing, and give the same result every run.

Run scripts and docs: app.py is a CLI (ask, eval --version 1/2 [--warm], demo, reset-memory; flags --sqlite, --report, --quiet). The README covers setup, commands, design and limitations. `python app.py demo` produced the screenshots: cold run (2 tool calls, 7.2 s), warm run (memory hit, 1 call, 3.7 s) and error recovery (read_policy ERROR → policy_search → read_policy ok). It also wrote results/demo_report.json (JSON run report extension).

Optional: SQLite checkpointing via LangGraph SqliteSaver (--sqlite). A run's state was saved across 8 checkpoints and reloaded by thread id.

Final note:

Fantastic work on reaching the final stage of this mini project!

You've taken on one of the most challenging and rewarding areas of modern applied AI—building a grounded, agentic Policy Assistant that plans, reasons, uses tools, and operates under explicit constraints. Through this project, you've moved beyond simple prompt-response systems and into the mindset of professionals who design reliable, auditable AI assistants for real organisational use.

This mini project asked you to think carefully about how an agent behaves, not just what it answers. From enforcing document grounding and citation rules, to handling recovery when tools fail, to managing step budgets through graph orchestration, each design decision reflected real-world trade-offs faced in enterprise AI systems. If parts of the build felt complex—memory write conditions, critic checks, or routing logic—that complexity is a sign you were working on authentic problems.

Remember that agentic systems improve through iteration. Debugging a missed citation, refining a prompt to enforce acceptance rules, or tuning graph stop conditions are not setbacks—they are the essence of learning how dependable AI systems are built. Each experiment you ran and each failure you analysed strengthened your understanding of controllability, safety, and evaluation.

As you review your results—cold vs warm runs, recovery behaviour, evaluation pass rates—focus on the insights you gained about grounding, memory, and orchestration. These skills form a strong foundation for building scalable, trustworthy assistants in real-world settings.

Keep experimenting, keep reasoning carefully, and take pride in the system you've built. The discipline and clarity you've developed here will carry forward into every advanced AI project you tackle next.

THE END

For learner use only